2-Bit Branch Prediction

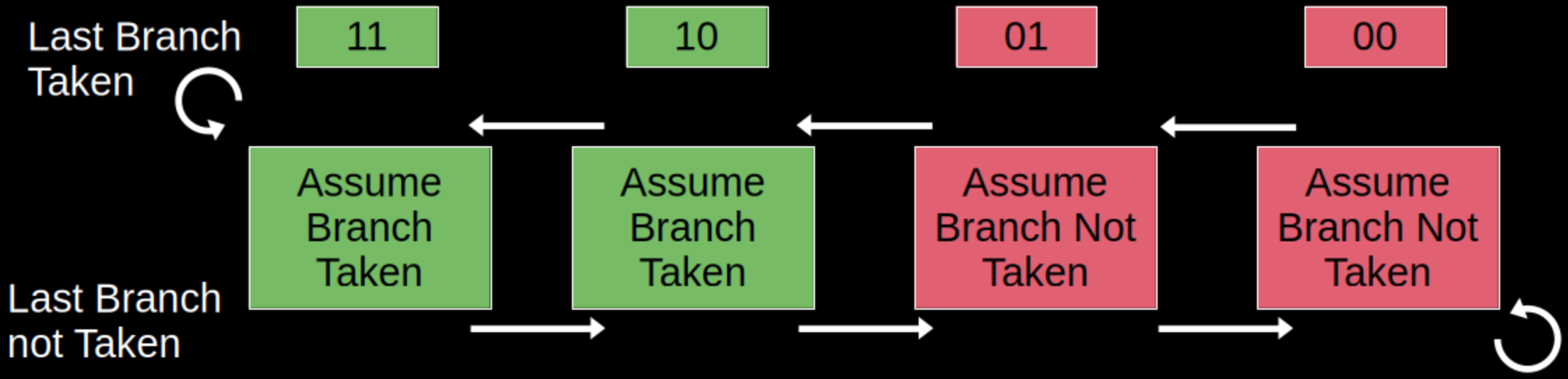
CS4200 Project Presentation

By Sylas Wilson

3 May, 2026

Introduction: 2-Bit Branch Prediction

- Branching affects performance: requiring stalls and/or pipeline flushes, decreasing instruction throughput efficiency
- Prediction models seek to decrease negative performance effects



Code: Decode Stage

```
next_id_ex["rs2_val"] = u32(regs[rs2])
next_id_ex["alu_op"] = alu_control(c, d)

#branch predictor if branch type
if c["Branch"] == 1:
    next_id_ex["Branch"] = 1
    if (u32(branch_predictor) >> 1) == 1:
        #predictor MSB is 1
        predictor_lines.append(f"Cycle {cycle} ID Stage: Branch Predic
        next_id_ex["pc_plus4"] = u32(if_id["pc"] + imm)
        next_id_ex["not_guessed_branch"] = u32(if_id["pc"] + 4)
    else: #MSB==0, guessing no branch, default to pc+4
        next_id_ex["not_guessed_branch"] = u32(if_id["pc"] + imm)
```

Code: Execution Stage

```
if c.get("Branch", 0) == 1 and c.get("BrType") is not None:
    branch_stats["branch_total"]+=1
    taken = branch_taken(c.get("BrType"), a, b_reg)
    #update branch predictor with actual branching result
    # for 2-bit alg update at end of cycle
    if taken:
        increment_prediction = "+"
    else:
        increment_prediction = "-"
    #predictor already chose a path. If they do not match, change next_pc to id_ex["not_guessed_branch"]
    if (taken and (branch_predictor >> 1 == 0)) or (not taken and (branch_predictor >> 1 == 1)):
        next_pc = id_ex["not_guessed_branch"]
        prediction_wrong = True
    else:
        next_pc = id_ex["pc_plus4"]
        branch_stats["correct_predictions"]+=1
```

Code: End of Cycle Update

```
#Branch predictor updates:
if increment_prediction == "+": #check operator
    if branch_predictor != 0b11: #check if maximum
        branch_predictor = branch_predictor + 0b01
    predictor_lines.append(f"Cycle {cycle}: Branch was taken. 2-bi
elif increment_prediction == "-": #check operator
    if branch_predictor != 0b00: #check if minimum
        branch_predictor = branch_predictor - 0b01
    predictor_lines.append(f"Cycle {cycle}: Branch was not taken.
else:
    predictor_lines.append(f"Cycle {cycle}: No branch instruction
```

Observations

- When adding functionality, know what fields can be used and what fields to add.
  - Control flow required: 1 boolean for control flow and 2 fields to id_ex
- Initial Implementation errors:
  - Bool added to control flow interrupted Jump instruction flush, adjusted in order to avoid interruptions in previous logic
  - Forgot to set new field "next_id_ex["Branch"] = 1" for checking if branch in EX stage

Citations

Fog, Agner. *The Microarchitecture of Intel, AMD and via CPUs an Optimization Guide for Assembly Programmers and Compiler Makers*.

GeeksforGeeks. “Correlating Branch Prediction.” *GeeksforGeeks*, 3 June 2020, www.geeksforgeeks.org/computer-organization-architecture/correlating-branch-prediction/. Accessed 2 May 2026.

Demonstration

Public GitHub: https://github.com/saesy/RISC-V-Simulator-in-Python

Used example code:

```
.word

addi x5,x0, 0 // counter
addi x6, x0, 0 // total
addi x7, x0, 10 // end condition

loop:
addi x6,x6, 3 //add to total
addi x5, x5, 1 //increment counter
bne x5,x7, loop //branch instruction

sub x29, x6, x7 // total - end condition
```

```
def silly(total, increment, loops):
    while (increment != loops):
        total+=3
    return total

x,i = 0
y=10
x = silly(x,i,y) # adds 3 ten times

z = x - y # 30-10
#z=20
```